

TP Chapitre 1 : Représentation Numérique

Exploration Pratique avec Compilation et Débogage

Dr. El Hadji Bassirou TOURÉ
Département de Mathématiques et Informatique
Université Cheikh Anta Diop de Dakar

Prérequis

Ce TP suppose que vous avez configuré votre environnement Docker (TP-0) et maîtrisez :

- Compilation avec `gcc` et options `-Wall -Wextra -g`
- Débogage de base avec `gdb`
- Utilisation des sanitizers (`-fsanitize=undefined`)

Commandes de référence

```
# Compilation standard
gcc -Wall -Wextra -Werror -g prog.c -o prog

# Compilation avec détection d'undefined behavior
gcc -Wall -Wextra -g -fsanitize=undefined prog.c -o prog

# Débogage
gdb ./prog
(gdb) break main
(gdb) run
(gdb) print variable
(gdb) print/x variable    # Affichage hexadécimal
(gdb) print/t variable    # Affichage binaire
```

1 Exercice 1 : Affichage Binaire et Hexadécimal

Objectif

Créer des fonctions d'affichage pour visualiser les représentations binaire et hexadécimale des entiers.

1.1 Créer un fichier `affichage.c` avec le code suivant :

```
1  #include <stdio.h>
2  #include <stdint.h>
3
4  // Affiche un entier 8 bits en binaire
5  void print_binary_8(uint8_t n) {
6      printf("0b");
7      for (int i = 7; i >= 0; i--) {
8          printf("%d", (n >> i) & 1);
9      }
10 }
11
12 int main() {
13     uint8_t a = 42;
14     printf("a = %3d = ", a);
15     print_binary_8(a);
16     printf(" = 0x%02X\n", a);
17
18     // TODO: Tester avec d'autres valeurs
19
20     return 0;
21 }
```

Questions :

- Compiler et exécuter. Vérifier que $42 = 00101010 = 0x2A$.
- Ajouter les tests pour : 0, 127, 128, 255.
- Écrire une fonction `print_binary_32(uint32_t n)` pour les entiers 32 bits.
- Utiliser GDB pour afficher la valeur de `a` avec `print/t a` et `print/x a`. Comparer avec votre fonction.

2 Exercice 2 : Exploration Signed vs Unsigned

Objectif

Observer les différences d'interprétation entre types signés et non signés.

2.1 Créer un fichier `signed_unsigned.c` :

```
1 #include <stdio.h>
2 #include <stdint.h>
3
4 int main() {
5     // Même motif binaire, interprétations différentes
6     uint8_t u = 200;
7     int8_t s = 200; // Attention: conversion implicite
8
9     printf("=== Même valeur initiale: 200 ===\n");
10    printf("uint8_t u = %u\n", u);
11    printf("int8_t s = %d\n", s);
12
13    // Afficher le motif binaire de chacun
14    printf("u en binaire: ");
15    for (int i = 7; i >= 0; i--) printf("%d", (u >> i) & 1);
16    printf("\n");
17
18    printf("s en binaire: ");
19    for (int i = 7; i >= 0; i--) printf("%d", ((uint8_t)s >> i) & 1);
20    printf("\n");
21
22    return 0;
23 }
```

Questions :

- Compiler avec `gcc -Wall -Wextra`. Y a-t-il un warning ? Pourquoi ?
- Quelle valeur affiche `s` ? Expliquer avec la formule B2T.
- Modifier le code pour tester les valeurs : 127, 128, 255, 0.
- Utiliser GDB : mettre un breakpoint après les affectations, puis `print u` et `print s`. Que montre GDB ?

3 Exercice 3 : Pièges de Comparaison

Objectif

Identifier et comprendre les bugs liés aux comparaisons signed/unsigned.

3.1 Créer un fichier `comparaisons.c` :

```
1 #include <stdio.h>
2 #include <stdint.h>
3
4 int main() {
5     int x = -1;
6     unsigned int y = 0;
7
8     printf("x = %d, y = %u\n", x, y);
9     printf("x < y ? %d\n", x < y);
10
11    // Prédire le résultat avant d'exécuter!
12    printf("-1 < 0U ? %d\n", -1 < 0U);
13    printf("-1 == 0xFFFFFFFF ? %d\n", -1 == 0xFFFFFFFF);
14
15    return 0;
16 }
```

Questions :

- Prédire les résultats AVANT de compiler. Puis vérifier.

- b) Compiler avec gcc -Wsign-compare. Quels warnings apparaissent ?
- c) Expliquer pourquoi $-1 < 0U$ est faux (quelle conversion a lieu ?).
- d) Ajouter un test : $2147483647U > -2147483648$. Prédire puis vérifier.
- e) En GDB, utiliser `print/u x` pour voir x interprété comme unsigned.

4 Exercice 4 : Détection d'Overflow avec Sanitizer

Objectif

Utiliser le sanitizer pour détecter les undefined behaviors liés aux overflows.

4.1 Créer un fichier overflow.c :

```

1 #include <stdio.h>
2 #include <stdint.h>
3
4 int main() {
5     // Test 1: Overflow unsigned (comportement D FINI)
6     printf("=== Overflow Unsigned ===\n");
7     uint8_t u = 255;
8     printf("u = %u\n", u);
9     u = u + 1;
10    printf("u + 1 = %u\n", u);
11
12    // Test 2: Overflow signed (UNDEFINED BEHAVIOR!)
13    printf("\n=== Overflow Signed ===\n");
14    int8_t s = 127;
15    printf("s = %d\n", s);
16    s = s + 1; // UB!
17    printf("s + 1 = %d\n", s);
18
19    return 0;
20 }
```

Questions :

- a) Compiler SANS sanitizer : `gcc -g overflow.c -o overflow`. Exécuter. Noter les résultats.
- b) Compiler AVEC sanitizer : `gcc -g -fsanitize=undefined overflow.c -o overflow`. Exécuter. Que se passe-t-il ?
- c) Le sanitizer détecte-t-il l'overflow unsigned ? Pourquoi ?
- d) Ajouter un test avec `int8_t s = -128; s = s - 1;`. Le sanitizer le détecte-t-il ?
- e) Compiler avec -O2 (optimisation) sans sanitizer. Le résultat change-t-il ?

5 Exercice 5 : Exploration IEEE 754

Objectif

Visualiser la structure interne des nombres flottants.

5.1 Créer un fichier ieee754.c :

```

1 #include <stdio.h>
2 #include <stdint.h>
3
4 // Union pour accéder aux bits d'un float
5 typedef union {
6     float f;
7     uint32_t u;
8 } Float32;
9
10 void analyze_float(float value) {
11     Float32 num;
12     num.f = value;
13
14     // Extraire les champs
15     uint32_t sign = (num.u >> 31) & 1;
16     uint32_t exp = (num.u >> 23) & 0xFF;
17     uint32_t frac = num.u & 0x7FFFFF;
18
19     printf("Valeur: %f\n", value);
20     printf("Hex: 0x%08X\n", num.u);
21     printf("Signe: %u\n", sign);
22     printf("Exposant brut: %u (biaisé)\n", exp);
23 }
```

```

23     printf("Exposant réel: %d\n", (int)exp - 127);
24     printf("Fraction: 0x%06X\n", frac);
25     printf("---\n");
26 }
27
28 int main() {
29     analyze_float(1.0f);
30     analyze_float(-1.0f);
31     analyze_float(12.5f);
32
33     // TODO: Ajouter d'autres tests
34
35     return 0;
36 }

```

Questions :

- Compiler et exécuter. Vérifier que $1.0f = 0x3F800000$.
- Ajouter l'analyse de : $0.5f$, $0.1f$, $-12.5f$.
- Pour $0.1f$, observer la fraction. Pourquoi n'est-elle pas "propre" ?
- Ajouter des tests pour les valeurs spéciales :
 - $1.0f / 0.0f$ (infini positif)
 - $0.0f / 0.0f$ (NaN)
 - $-0.0f$ (zéro négatif)
- En GDB, afficher `num.u` avec `print/t num.u` pour voir les 32 bits.

6 Exercice 6 : Comparaison de Flottants

Objectif

Comprendre pourquoi la comparaison directe de flottants est dangereuse.

6.1 Créer un fichier `float_compare.c` :

```

1  #include <stdio.h>
2  #include <math.h>
3
4  int main() {
5      float a = 0.1f + 0.2f;
6      float b = 0.3f;
7
8      printf("a = %.20f\n", a);
9      printf("b = %.20f\n", b);
10     printf("a == b ? %d\n", a == b);
11     printf("a - b = %.20e\n", a - b);
12
13     // Comparaison correcte avec epsilon
14     float epsilon = 1e-6f;
15     printf("fabsf(a-b) < epsilon ? %d\n", fabsf(a - b) < epsilon);
16
17     // Test: associativité
18     float x = (3.14f + 1e10f) - 1e10f;
19     float y = 3.14f + (1e10f - 1e10f);
20     printf("\nx = (3.14 + 1e10) - 1e10 = %f\n", x);
21     printf("y = 3.14 + (1e10 - 1e10) = %f\n", y);
22     printf("x == y ? %d\n", x == y);
23
24     return 0;
25 }

```

Questions :

- Compiler avec `gcc -Wall -g -lm float_compare.c -o float_compare`. Exécuter.
- Expliquer pourquoi `a != b` alors que mathématiquement $0.1 + 0.2 = 0.3$.
- Quelle est la différence entre `x` et `y` ? Pourquoi ?
- En GDB, mettre un breakpoint après le calcul de `a` et `b`, puis utiliser `print a` et `print b`. GDB montre-t-il la différence ?

7 Exercice 7 : Projet Mini-Convertisseur

Objectif

Créer un programme interactif de conversion entre bases.

7.1 Créer un programme `convertisseur.c` qui :

1. Demande à l'utilisateur un nombre (décimal, hexadécimal avec préfixe 0x, ou binaire avec préfixe 0b)
2. Affiche ce nombre dans les trois bases
3. Affiche l'interprétation signed et unsigned (sur 8, 16, ou 32 bits selon la valeur)

Exemple d'exécution attendue :

```
Entrez un nombre (dec, 0xHEX, ou 0bBIN): 0xFF
=== Résultat sur 8 bits ===
Décimal unsigned: 255
Décimal signed:   -1
Hexadécimal:      0xFF
Binaire:          11111111
```

```
Entrez un nombre (dec, 0xHEX, ou 0bBIN): -42
=== Résultat sur 8 bits ===
Décimal unsigned: 214
Décimal signed:   -42
Hexadécimal:      0xD6
Binaire:          11010110
```

Conseils :

- Utiliser `strtol()` avec différentes bases pour parser l'entrée
- Créer des fonctions séparées pour chaque type d'affichage
- Tester avec GDB pour vérifier les conversions intermédiaires

8 Soumission

À rendre

- Tous les fichiers source (.c)
- Un fichier `README.md` avec :
 - Réponses aux questions de chaque exercice
 - Captures d'écran montrant l'utilisation de GDB et du sanitizer
 - Explications des bugs/comportements observés

Format : Archive `TP1_NomPrenom.tar.gz`